Class 6 : R functions

Omar (PID: A19093654)

Table of contents

Background

All functions in R have at least 3 things:

- a *name* (we picked that and use it to call the function)
- input *arguments* (one or more commas separated inputs that go inside the brackets when we call the function
- the *body* (the lines of R code that do work of the function)

A first function

Here we will create a new function to add some numbers. Let's call it 'add()'

Input arguments can be either "required" or "optional", the latter have fall back default values that will be used if the user doesn't specify them.

' ' '[r] add <- function(x, y) { x + y }

Q1A:

```
add <- function (x, y=1000) {
  x + y
}
```

Can we use our new function

```
add(10, 100)
```

```
[1] 110
```

Q1B:

```
add = function(x, y=0) {
 sum(x,y)
}
```

We can explicitly set a **return** value output from a fnuction (rather than the default last line) by using the 'return()' function.

Q2

A useful function here is that "base R" 'sample()' function:

```
# 1. Background check
sample(1:5, size=3)
```

```
[1] 5 2 4
```

```
# 2. DEFINE the basic function here so R knows what it is
generate_dna <- function(len = 10) {
  sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE)
}

# 3. Now it is safe to CALL the function
generate_dna(len=100)
```

```
  [1] "G" "T" "G" "T" "A" "G" "T" "C" "G" "A" "T" "A" "T" "G" "G" "A" "A" "A"
 [19] "G" "C" "T" "A" "C" "G" "G" "A" "T" "C" "C" "A" "C" "C" "C" "T" "C" "G"
 [37] "G" "T" "T" "G" "T" "T" "C" "G" "G" "C" "T" "G" "C" "G" "C" "T" "G" "C"
 [55] "G" "A" "A" "G" "T" "A" "A" "A" "A" "T" "G" "C" "T" "A" "C" "C" "G" "A"
 [73] "G" "T" "C" "G" "T" "G" "C" "T" "G" "G" "A" "C" "T" "G" "A" "C" "C" "T"
 [91] "G" "G" "T" "G" "T" "C" "C" "G" "A" "G"
```

We can use this to make a random nucleotide sequence if we draw from "A", "C", "G", and "T".

```
generate_dna(len=100)
```

```
  [1] "A" "T" "G" "C" "T" "G" "C" "C" "C" "C" "T" "T" "G" "A" "C" "C" "T" "C"
 [19] "C" "G" "C" "G" "C" "C" "C" "T" "G" "G" "T" "A" "A" "C" "A" "C" "C" "C"
 [37] "C" "T" "C" "C" "A" "G" "A" "G" "G" "A" "G" "T" "C" "G" "A" "G" "G" "G"
 [55] "A" "G" "G" "A" "G" "G" "A" "G" "T" "G" "T" "T" "G" "C" "T" "G" "T" "A"
 [73] "A" "A" "A" "T" "A" "A" "G" "G" "C" "A" "G" "T" "C" "A" "G" "C" "T" "A"
 [91] "A" "T" "G" "A" "C" "T" "G" "G" "C" "T"
```

Q2A:

Defining the function before calling it

```
generate_dna <- function(len = 10) { sample(x = c("A", "C", "G", "T"), size = len, replace =

generate_dna(len=10)
```

```
 [1] "A" "C" "T" "G" "T" "G" "A" "C" "G" "T"
```

```
# Since we defined it above, this call works now
generate_dna(len=10)
```

```
 [1] "G" "T" "A" "C" "A" "T" "T" "G" "A" "T"
```

what was wrong was that the size and len weren't linked up (=).

Q2b:

```
generate_dna <- function(len = 10, single.element = FALSE) {
  ans <- sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE)

  if (single.element == TRUE) {
    ans <- paste(ans, collapse = "")
  }
  return(ans)
}

# Test the new argument
generate_dna(len = 10, single.element = TRUE)
```

```
[1] "GGTTGGAATG"
```

Functions that might help you are 'paste()', 'if()', 'cat()', and 'return()'.

Q2c:

```
generate_dna_fasta <- function(len = 10) {
  ans <- sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE)
  seq <- paste(ans, collapse = "")

  # cat() combined with \n prints the required FASTA lines [cite: 420]
  cat(">len", len, "\n", seq, "\n", sep="")
}

# Example of the final behavior
generate_dna_fasta(9)
```

```
>len9
GGTTGGATG
```

Q3:

```
generate_protein <- function(len = 6) {
  amino_acids <- c("A", "R", "N", "D", "C", "Q", "E", "G", "H", "I",
                   "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V")

  raw_sequence <- sample(x = amino_acids, size = len, replace = TRUE)
  protein_string <- paste(raw_sequence, collapse = "")

  return(protein_string)
}

generate_protein(6)
```

```
[1] "GWKTCM"
```

Q4:

```
for (i in 6:13) {
  protein_seq <- generate_protein(i)
  cat(">id.", i, "\n", protein_seq, "\n", sep = "")
}
```

```
>id.6
PGGWPP
>id.7
KTHDGFI
>id.8
IPKSEEQS
>id.9
KNVMKFFYS
>id.10
VVKTHMYPED
>id.11
WRWEQWDLCYW
>id.12
VNFHDDQWLWMQ
>id.13
DTQGIYRNYIDNV
```

Q5:

legnth	best hit % identity	best hit % coverage	unique? (y/n)
6	100	100	N
7	100	100	N
8	100	88	Y
9	100	89	Y
10	100	80	Y
11	100	89	Y
12	81	92	Y
13	90	85	Y

Q5A: My generated peptides start looking "unique in nature" at a sequence length of 8 amino acids. This is the first point in the table where a hit fails to meet the definition of 100% identity and 100% coverage.

Q5B: Short random peptides are almost always found in the nr database because the sequence space (20^L) is significantly smaller than the known protein universe . For a peptide of length 6, there are only around 64 million (20^6) possible combinations, making chance matches likely. As L increases, the sequence space grows exponentially, making it statistically improbable for a random sequence to match an existing natural protein.

```
cat("hello \n there")
```

```
hello
 there
```

Q6:

Based on the data from Q5, the minimum length for MHC class II peptide presentation is likely around 8 or 9 amino acids. My results show that peptides of length 6 and 7 are not unique in nature, meaning they match existing proteins with 100% identity and coverage. If the immune system presented very short peptides, it would be a bad design choice because these sequences are so common that the system could not reliably distinguish between "self" and "non-self" pathogens. This lack of specificity would likely lead to autoimmune attacks on the body's own tissues. By requiring longer peptides, where the sequence space (20^L) is much larger, the immune system ensures the peptides are unique enough to trigger an accurate immune response.